

R Programming

Suprit Saha
Tutorial Five-Six

March 22, 2015

1 Descriptive Statistics

Once our data is properly organized, the next step will typically be to **describe the distribution** of each variable numerically, followed by an exploration of the **relationships** among selected variables two at a time.

Let us consider the internal dataset **mtcars** in the **datasets** package

```
1 library(datasets)
2 data(mtcars)
3 mtcars
```

- Via **summary()**

```
1 vars <- c("mpg", "hp", "wt") #Considering only three variables
2 summary(mtcars[vars])
```

Note: The **summary()** function provides the **minimum**, **maximum**, **quartiles**, and the **mean** for numerical variables and **frequencies** for factors and logical vectors

- Via **stat.desc()** in the **pastecs** package

```
1 library(pastecs)
2 stat.desc(mtcars[vars])
```

Note: The number of values, null values, missing values, minimum, maximum, range, sum median, mean, standard error of the mean, 95 percent confidence interval for the mean, variance, standard deviation, and coefficient of variation are provided

- Via **describe()** in the **Hmisc** package()

```
1 library(Hmisc)
2 describe(mtcars[vars])
```

Note: Shows number of variables and observations, the number of missing and unique values, the mean, quantiles, and the five highest and lowest values

2 Descriptive statistics by group

- Via `aggregate()`

```
1 aggregate(mtcars[vars], by=list(am=mtcars$am), mean)  
2 aggregate(mtcars[vars], by=list(am=mtcars$am), sd)
```

Note: When we aggregate data we replace observations with summary statistics based on those observations. Unfortunately, `aggregate()` only allows us to use single value functions such as mean, standard deviation etc. It won't return several statistics at once

- Via `by()`

```
1 dstats <- function(x)(c(mean=mean(x), sd=sd(x))) #user defined function  
2 by(mtcars[vars], mtcars$am, dstats)
```

Note: Can return several statistics at once

- Via split-lapply-sapply combination

```
1 z<-split(x = mtcars[vars],list(am=mtcars$am)) #Splitting dataset by am variable(factor)  
2 lapply(z,function(y) sapply(y,function(x) c(mean=mean(x),sd=sd(x)))))
```

Note: Most built in functions are internally split-apply-lapply combinations

3 Frequency & Contingency tables

The alternative to using graphics is to summarize your data in tabular form. Broadly speaking, if you want to convey *detail* use a table, and if you want to show *effects* then use graphics. You are more likely to want to use a table to summarize data when your explanatory variables are categorical (such as peoples names, or different commodities) than when they are continuous (in which case a scatterplot is likely to be more informative)

Let us consider the **Arthritis** dataset in **vcd** package

```
1 library(vcd)
2 data(Arthritis)
3 Arthritis
```

- **Generating Frequency tables**

R provides several methods for creating frequency and contingency tables. The most important functions are listed below

Function	Description
<code>table(var1, var2, ..., varN)</code>	Creates an N-way contingency table from N categorical variables (factors)
<code>xtabs(formula, data)</code>	Creates an N-way contingency table based on a formula and a matrix or data frame
<code>prop.table(table, margins)</code>	Expresses table entries as fractions of the marginal table defined by the <i>margins</i>
<code>margin.table(table, margins)</code>	Computes the sum of table entries for a marginal table defined by the <i>margins</i>
<code>addmargins(table, margins)</code>	Puts summary <i>margins</i> (sums by default) on a table
<code>ftable(table)</code>	Creates a compact "flat" contingency table

Figure 1: Important Table Functions

- **One way table**

```
1 mytable <- with(Arthritis, table(Improved)) # One way table
2 mytable <- xtabs(~Improved, data=Arthritis) # Alternative
3 prop.table(mytable) # Proportions
4 prop.table(mytable)*100 #Percentages
```

– Two way table

```
1 mytable <- with(Arthritis, table(Treatment, Improved))
2 mytable <- xtabs( ~Treatment+Improved, data=Arthritis) # Alternative
3 margin.table(mytable, 1) # Row total
4 prop.table(mytable, 1) # Row proportions
5 margin.table(mytable, 2) # Column total
6 prop.table(mytable, 2) # Column proportions
7 prop.table(mytable) # Cell proportions
8 addmargins(prop.table(mytable, 1), 2) # Sum column
9 addmargins(prop.table(mytable, 2), 1) # Sum row
```

Note: The `table()` function ignores missing values (NAs) by default. To include NA as a valid category in the frequency counts, include the `table` option `useNA= "ifany"`

4 Programming Structures

R is a **block-structured language** like C, C++, Python etc. In the normal course of events, the statements in an R program are executed sequentially from the top of the program to the bottom. But there are times that you'll want to execute some statements repetitively, while only executing other statements if certain conditions are met. This is where control-flow constructs come in.

R has the standard control structures you'd expect to see in a modern programming language. For the syntax examples throughout this section, keep the following in mind:

- **statement** is a single R statement or a compound statement (a group of R statements enclosed in curly braces { } and separated by semicolons).
- **cond** is an expression that resolves to true or false.
- **expr** is a statement that evaluates to a number or character string.
- **seq** is a sequence of numbers or character strings.

4.1 Repetition and looping

Looping constructs repetitively execute a statement or series of statements until a condition isn't true. These include the *for* and *while* structures.

- The `for()` loop

for (var in seq) statement

```
1 for(i in 1:10) print("R") # R is printed 10 times
2 x <- c(5, 12, 13)
3 for(n in x) print(n^2) # Output 25,144,169
```

Note: The *for* loop executes a statement repetitively until a variable's value is no longer contained in the sequence *seq*

- The `while()` loop

while (cond) statement

```
1 i <- 10
2 while (i > 0)
3 {
4   print("R")
5   i <- i - 1
6 } # R is printed 10 times
```

Note: A *while* loop executes a statement repetitively until the condition is no longer true. Make sure that the statements inside the brackets modify the while condition so that sooner or later it's no longer true, otherwise the loop will never end!

4.2 Conditional Execution

In conditional execution, a statement or statements are only executed if a specified condition is met. These constructs include *if-else*, *ifelse*, and *switch*.

- The `if-else()` loop

if (cond) statement
if (cond) statement1 else statement2

```
1 x <- c(2, 4, 7, 9, 11, 3, 2, 4)
2 for(i in 1:length(x))
3 {
4   if(x[i]%%2==0)
5   {
6     print("Even")
7   } else
8   {
9     print("Odd")
10  }
11 } # prints even or odd depending on number
```

Note: The *if-else* control structure executes a statement if a given condition is true. Optionally, a different statement is executed if the condition is false.

- The `ifelse()` loop

`ifelse(cond, statement1, statement2)`

```
1 x <- c(2, 4, 7, 9, 11, 3, 2, 4)
2 ifelse(x%%2==0,"Even","Odd")
3 [1] "Even" "Even" "Odd" "Odd" "Odd" "Odd" "Even" "Even" #Output
```

Note: The *`ifelse`* construct is a compact and vectorized version of the if-else construct

- The `switch()` loop

`switch(expr, ...)`

where ... represents statements tied to the possible outcome values of `expr`

```
1 feelings <- c("sad", "afraid")
2 for (i in feelings)
3   print(
4     switch(i,
5       happy = "I am glad you are happy",
6       afraid = "There is nothing to fear",
7       sad = "Cheer up",
8       angry = "Calm down now"
9     )
10  )
11 [1] "Cheer up"
12 [1] "There is nothing to fear" #Output
```

Note: *`switch`* chooses statements based on the value of an expression

4.3 User defined Functions

One of R's greatest strengths is the user's ability to add functions. In fact, many of the functions in R are functions of existing functions. The structure of a function looks like this:

```
myfunction <- function(arg1, arg2, ... )
{
  statements
  return(object)
}
```

```

1  add<-function(x,y) # Function to add any two vectors
2  {
3    x+y
4  }
5  add(c(2,4),c(3,4)) #Output 5,8
6
7  even<-function(x) # Function to print only even nos
8  {
9    return(x[x%%2==0])
10 }
11 even(1:10)
12 [1] 2 4 6 8 10 #Output

```

Note: Functions in R are simply objects of a particular type (of mode "function"). Like other objects, they can be assigned to variables. Most of the time, they actually are assigned to a variable, and we refer to the function by the name of the variable it is assigned to. All standard functions (like print(), plot(), c()) are actually variables, whose value is an R object of mode "function". When we refer to the seq() function, we actually mean the value of the variable seq.

Functions in R= Built-in +User defined

5 R Graphics

R has a very rich set of graphics facilities compared to other statistical softwares. R includes at least **three** graphical systems, the standard **graphics** package, the **lattice** package for Trellis graphs and the grammar-of-graphics **ggplot2** package.

Three references: (already shared with you)

- R Graphics -Peter Murrell
- Lattice: Multivariate Data Visualization with R -Deepayan Sarkar
- ggplot2:Elegant Graphics for Data Analysis - Hadley Wickham